Programming Assignment 4
Nathan Lenahan
10/15/26

Analysis of the two GenAI Programs:

a) The two GenAI programs used to generate the two programs were Claude and Gemini
b) Both Claude and Gemini were accessed through their respective websites using Google Chrome

c) The following prompt was used for both GenAI programs:

CREATE THE FOLLOWING PROGRAM USING C++ CODE. YOU MUST USE OBJECTS, NOT FUNCTIONS, TO IMPLEMENT THE FOLLOWING REQUIREMENTS.

OVERVIEW:

- Sudoku is a number puzzle in which a 9x9 grid is subdivided into 9 3x3 subgrids, or blocks.
- Each of the cells in the puzzle must be filled in with a single digit 1-9, subject to the following constraints:
    - Each digit from 1 to 9 must appear exactly once in each row.
    - Each digit from 1 to 9 must appear exactly once in each column.
    - Each digit from 1 to 9 must appear exactly once in each block.
- Published puzzles are usually constructed to have a unique solution. This requires that at least 17 cells be filled in (and may require more; depending on the puzzle and which cells are empty, as many as 78 filled cells can be required to guarantee uniqueness). In general, the number of cells that are already filled in is not a particularly good indicator of how difficult a Sudoku puzzle is to solve.
- The Sudoku problem, then, is this: Given a partially completed Sudoku puzzle, find the (or at least a) solution; if no solution exists, prove that.
- The approach used should implement a brute-force search, in which we exhaustively try possibilities until we find a solution. This will be slower to run but is easier to solve.
    - Here's the idea: Suppose we have a Sudoku puzzle in which k of the 81 cells are unfilled. We identify an empty cell and deduce a list of candidates for what might go into that cell. If there are no such values (all digits are accounted for elsewhere in the same row, column, or grid), then we must have made a mistake in filling in the k cells and return a no-solution-found result. Otherwise, we try putting the first candidate value into the empty cell, and see if we can find a solution with the k+1 cells filled (i.e. a recursive call). If we get a no-solution-found to the first candidate, then we try the next, and so on, until we either find a solution (k == 81) or determine that no solution exists (all candidates report no solution, which again implies we took a wrong path somewhere earlier).

- You are given five files (puzzle1.txt through puzzle5.txt), each containing a Sudoku puzzle. Digits already filled in have been placed, and an underscore character (_) is used to indicate a blank square.
- Write a **recursive object-oriented program** to find the solution to the five puzzle files.
- The output for each solution should include:
    - The puzzle file name (e.g., puzzle1.txt)
    - The puzzle from the file printed out.
    - The solution to the puzzle printed out, or if no solution is found, “No solution found” printed out.
- The solution may not be unique; if there is more than one solution to the puzzle, the program should print them all out.

d) The following is the generated code from both GenAI programs:

CLAUDE - Raw Generated Code

```
// Sudoku brute-force solver (object-oriented, recursive backtracking)
// Compile: g++ -std=c++11 -o sudoku sudoku.cpp
// Run:     ./sudoku          (reads puzzle1.txt ... puzzle5.txt from the current directory)

#include <iostream>
#include <fstream>
#include <string>
#include <vector>
#include <cctype>

using namespace std;

// ---------------------------------------------------------------------------
// Board: holds a 9x9 grid and knows how to load, print, and validate placements
// ---------------------------------------------------------------------------
class Board {
public:
    static const int SIZE = 9;
    static const int BLOCK = 3;

    Board() {
        for (int r = 0; r < SIZE; r++)
            for (int c = 0; c < SIZE; c++)
                cells[r][c] = 0;
    }

    // Reads a puzzle from a file. Digits 1-9 are givens; '_' (or '0' / '.') is blank.
    // Whitespace is ignored, so both "53__7____" and "5 3 _ _ 7 ..." layouts work.
    bool load(const string& filename) {
        ifstream in(filename.c_str());
        if (!in) return false;
```

```
    int count = 0;
    char ch;
    while (count < SIZE * SIZE && in.get(ch)) {
        if (isspace(static_cast<unsigned char>(ch))) continue;
        int r = count / SIZE;
        int c = count % SIZE;
        if (ch >= '1' && ch <= '9') cells[r][c] = ch - '0';
        else                        cells[r][c] = 0;   // '_', '0', '.', anything else = blank
        count++;
    }
    return count == SIZE * SIZE;
}

int  get(int r, int c) const { return cells[r][c]; }
void set(int r, int c, int v) { cells[r][c] = v; }

// Finds the first empty cell (row-major order). Returns false if the board is full.
bool findEmpty(int& row, int& col) const {
    for (int r = 0; r < SIZE; r++)
        for (int c = 0; c < SIZE; c++)
            if (cells[r][c] == 0) {
                row = r;
                col = c;
                return true;
            }
    return false;
}

// True if value v does not already appear in the row, column, or block of (row, col).
bool canPlace(int row, int col, int v) const {
    for (int i = 0; i < SIZE; i++) {
        if (cells[row][i] == v) return false;   // row
        if (cells[i][col] == v) return false;   // column
    }
    int br = (row / BLOCK) * BLOCK;
    int bc = (col / BLOCK) * BLOCK;
    for (int r = br; r < br + BLOCK; r++)
        for (int c = bc; c < bc + BLOCK; c++)
            if (cells[r][c] == v) return false; // block
    return true;
}

// Checks that the given clues don't already violate any rule.
bool givensAreConsistent() {
    for (int r = 0; r < SIZE; r++)
        for (int c = 0; c < SIZE; c++) {
            int v = cells[r][c];
            if (v == 0) continue;
            cells[r][c] = 0;                    // temporarily remove to test against the rest
```

```
            bool ok = canPlace(r, c, v);
            cells[r][c] = v;
            if (!ok) return false;
        }
    return true;
}

void print(ostream& out) const {
    for (int r = 0; r < SIZE; r++) {
        if (r > 0 && r % BLOCK == 0) out << "------+-------+------\n";
        for (int c = 0; c < SIZE; c++) {
            if (c > 0 && c % BLOCK == 0) out << "| ";
            if (cells[r][c] == 0) out << "_";
            else                  out << cells[r][c];
            out << (c < SIZE - 1 ? " " : "");
        }
        out << "\n";
    }
}

private:
    int cells[SIZE][SIZE];
};

// ---------------------------------------------------------------------------
// SudokuSolver: recursive brute-force search that collects ALL solutions
// ---------------------------------------------------------------------------
class SudokuSolver {
public:
    explicit SudokuSolver(const Board& puzzle) : board(puzzle) {}

    // Runs the search and returns every solution found.
    const vector<Board>& solveAll() {
        solutions.clear();
        if (board.givensAreConsistent())
            search();
        return solutions;
    }

private:
    Board board;
    vector<Board> solutions;

    // Recursive step: pick an empty cell, try each candidate, recurse, then undo.
    void search() {
        int row, col;
        if (!board.findEmpty(row, col)) {   // no empty cells left -> all 81 filled
            solutions.push_back(board);
            return;
```

```
        }

        for (int v = 1; v <= 9; v++) {
            if (board.canPlace(row, col, v)) {
                board.set(row, col, v);     // try candidate
                search();                   // recurse with k+1 cells filled
                board.set(row, col, 0);     // backtrack
            }
        }
        // If no candidate worked, we simply return: this path is a dead end.
    }
};

// ---------------------------------------------------------------------------
// PuzzleRunner: loads one puzzle file, solves it, and reports the results
// ---------------------------------------------------------------------------
class PuzzleRunner {
public:
    explicit PuzzleRunner(const string& file) : filename(file) {}

    void run() {
        cout << "==============================\n";
        cout << "Puzzle file: " << filename << "\n";
        cout << "==============================\n";

        Board puzzle;
        if (!puzzle.load(filename)) {
            cout << "Error: could not read a complete 9x9 puzzle from " << filename << "\n\n";
            return;
        }

        cout << "Puzzle:\n";
        puzzle.print(cout);
        cout << "\n";

        SudokuSolver solver(puzzle);
        const vector<Board>& solutions = solver.solveAll();

        if (solutions.empty()) {
            cout << "No solution found\n\n";
            return;
        }

        if (solutions.size() == 1) {
            cout << "Solution:\n";
            solutions[0].print(cout);
            cout << "\n";
        } else {
            cout << solutions.size() << " solutions found:\n\n";
```

```
            for (size_t i = 0; i < solutions.size(); i++) {
                cout << "Solution " << (i + 1) << ":\n";
                solutions[i].print(cout);
                cout << "\n";
            }
        }
    }

private:
    string filename;
};

int main() {
    for (int i = 1; i <= 5; i++) {
        PuzzleRunner runner("puzzle" + to_string(i) + ".txt");
        runner.run();
    }
    return 0;
}
```

GEMINI - Raw Generated Code

```
#include <iostream>
#include <fstream>
#include <string>
#include <vector>

// Class representing a 9x9 Sudoku board and its solving logic
class SudokuBoard {
private:
    char grid[9][9];

    // Helper to check if placing 'num' at (row, col) is valid
    bool isValid(int row, int col, char num) const {
        // Check row
        for (int j = 0; j < 9; ++j) {
            if (grid[row][j] == num) {
                return false;
            }
        }
        // Check column
        for (int i = 0; i < 9; ++i) {
            if (grid[i][col] == num) {
                return false;
            }
        }
        // Check 3x3 subgrid (block)
        int startRow = row - (row % 3);
        int startCol = col - (col % 3);
        for (int i = 0; i < 3; ++i) {
```

```
            for (int j = 0; j < 3; ++j) {
                if (grid[startRow + i][startCol + j] == num) {
                    return false;
                }
            }
        }
        return true;
    }

    // Find the next empty cell marked with '_'
    bool findEmptyCell(int& row, int& col) const {
        for (row = 0; row < 9; ++row) {
            for (col = 0; col < 9; ++col) {
                if (grid[row][col] == '_') {
                    return true;
                }
            }
        }
        return false;
    }

public:
    SudokuBoard() {
        clear();
    }

    // Reset the board to blanks
    void clear() {
        for (int i = 0; i < 9; ++i) {
            for (int j = 0; j < 9; ++j) {
                grid[i][j] = '_';
            }
        }
    }

    // Load puzzle from a text file
    bool loadFromFile(const std::string& filename) {
        std::ifstream file(filename);
        if (!file.is_open()) {
            return false;
        }
        char ch;
        for (int i = 0; i < 9; ++i) {
            for (int j = 0; j < 9; ++j) {
                if (!(file >> ch)) {
                    return false;
                }
                if ((ch >= '1' && ch <= '9') || ch == '_') {
                    grid[i][j] = ch;
```

```
            } else {
                --j; // Retry if unexpected character encountered
            }
        }
    }
    return true;
}

// Print the puzzle grid cleanly
void print() const {
    for (int i = 0; i < 9; ++i) {
        if (i > 0 && i % 3 == 0) {
            std::cout << "------+-------+------\n";
        }
        for (int j = 0; j < 9; ++j) {
            if (j > 0 && j % 3 == 0) {
                std::cout << "| ";
            }
            std::cout << grid[i][j] << ' ';
        }
        std::cout << '\n';
    }
}

// Recursive brute-force method to find and print all solutions
void solveAll(int& solutionCount) {
    int row, col;
    // If no empty cells remain, a solution is found
    if (!findEmptyCell(row, col)) {
        solutionCount++;
        std::cout << "Solution " << solutionCount << ":\n";
        print();
        std::cout << "\n";
        return; // Return to continue searching for alternative solutions (backtracking)
    }

    // Try candidates '1' through '9'
    for (char num = '1'; num <= '9'; ++num) {
        if (isValid(row, col, num)) {
            grid[row][col] = num;       // Place candidate
            solveAll(solutionCount);    // Recursive call
            grid[row][col] = '_';       // Backtrack
        }
    }
}
};

// Application controller class managing puzzle files
class SudokuApp {
```

```
public:
    void run() {
        std::string puzzleFiles[5] = {
            "puzzle1.txt",
            "puzzle2.txt",
            "puzzle3.txt",
            "puzzle4.txt",
            "puzzle5.txt"
        };

        for (int i = 0; i < 5; ++i) {
            std::cout << "========================================\n";
            std::cout << "Puzzle File: " << puzzleFiles[i] << "\n";
            std::cout << "========================================\n";

            SudokuBoard board;
            if (!board.loadFromFile(puzzleFiles[i])) {
                std::cout << "Error: Could not open or read " << puzzleFiles[i] << "\n\n";
                continue;
            }

            std::cout << "Initial Puzzle:\n";
            board.print();
            std::cout << "\n--- Solutions ---\n\n";

            int solutionCount = 0;
            board.solveAll(solutionCount);

            if (solutionCount == 0) {
                std::cout << "No solution found.\n\n";
            } else {
                std::cout << "Total solutions found for " << puzzleFiles[i] << ": " << solutionCount << "\n\n";
            }
        }
    }
};

int main() {
    SudokuApp app;
    app.run();
    return 0;
}
```

e) <u>The following is the analysis and comparison of the two GenAI programs:</u>

Correctness:

When both programs were tested, both compiled and ran successfully. Both programs provided correct results for each of the five provided puzzles, and the solution for puzzle 1 matched the

given solution1.txt file. The formatting of the output was also correct for both programs; each printed the filename, original puzzle, and solution(s) or no solution message.

The requirements were followed accurately for both programs. They each correctly use objects and recursive backtracking to solve the puzzles. Claude uses Board, SudokuSolver, and PuzzleRunner classes, and Gemini uses just a SudokuBoard class and SudokuApp class. Both are valid approaches, but the difference is something important to note. Each program uses the same process of finding an empty cell, checking the possible digits against the row, column, and block, and then recursively continuing with an allowed digit. The program then removes the digit that was attempted after the recursive call. This allows more digit possibilities to be explored. After a solution is found, both correctly keep searching for more solutions to ensure that all solutions are accounted for.

A difference that was noted was the way each program handles conflicting starting digits. Claude uses a givensAreConsistent() method that checks if the original clues violate any rules before the search is started. Gemini uses its isValid() method, but does not check the original clues. When each was tested with a completely filled board with conflicting digits, Claude rejected the puzzle while Gemini incorrectly printed it as a solution. This is because Gemini just assumes that a board without any empty cells is solved, and doesn't check to make sure the starting digits are valid first.

Both programs handle file validation in their own ways. Claude treats any non-whitespace character other than digits 1 through 9 as an empty cell. Gemini skips any unexpected characters and continues through until the board is filled. Both programs also don't check for extra content after the first 81 cells. Both programs are able to detect when a file is unable to be opened or doesn't have enough cells, but neither fully validates the format of the input file. Overall, both successfully printed the correct results, but Claude has the upper hand with its ability to handle conflicting starting clues in a more reliable way.

Execution Time:

Both programs use a recursive search with backtracking, so the execution time depends on how many possible placements for digits need to be explored. For every empty cell, digits 1 through 9 are tried and continue recursively whenever a digit is allowed. If a placement leads to a dead end, both programs backtrack to try another digit. The worst-case time complexity of both programs can be analyzed as O(9^m), where m is the number of empty cells at the beginning of a given puzzle. This represents up to nine possibilities for each empty cell in a puzzle.

Claude uses findEmpty() to locate the next cell that is empty and canPlace() to check to ensure that a candidate digit doesn't conflict with the row, column, or block of that cell. Gemini does the same with its own defined methods, findEmptyCell() and isValid(). Both use the same search path method of searching for an empty cell in row order and trying candidate digits in ascending order. Since the board is guaranteed to be 9x9, the act of searching for an empty cell and checking placement takes a bounded amount of work. The execution time is also impacted by

the amount of searching the program still does after a solution is found, since it looks for other possible solutions.

There are some small differences in the work that each program does. Claude does an initial check of the clues before searching, which adds a small bit to the initial cost, but it makes it so it can avoid searching a puzzle that already contains conflicting digits. Claude copies each completed board into a vector while Gemini prints each solution immediately. This means Gemini avoids having to store and reallocate a vector of completed boards. These differences don't change the overall complexity of the program, but they affect the amount of necessary work done outside of the search. Both programs ran quickly when tested, but neither had a big noticeable speed advantage over the other.

Space Complexity:

Both programs store a fixed 9x9 board and require memory usage for recursive calls during backtracking. One empty cell is filled before continuing for each recursive step of the board, so the depth of the recursion depends on how many initial empty cells there are, m. This means the recursion stack has a space complexity of O(m). Both programs also modify the current board and undo placements when backtracking instead of creating a whole new board for each recursive call.
The main difference between the two is the way that completed solutions are handled. Claude stores each completed puzzle in a vector of Board objects. This can require up to O(s) additional space, with s being the number of solutions. This is because each stored board contains a fixed 81 cells. Combine this with the recursion stack space complexity, and it is a total of O(m+s) space complexity for Claude's implementation. It's possible that the vector may have unused capacity as it grows, but this doesn't affect the space complexity.

Gemini prints each solution immediately instead of storing completed boards for access later. It only needs the current board, recursion stack, and a solution counter, which gives the program an overall space complexity of O(m). This is an important distinction since the O(s) space complexity that Claude has is completely omitted in Gemini's implementation. Both programs process the five puzzles one at a time, so neither program stores all five puzzles and their results in memory at once. Overall, Gemini's memory usage is more efficient when a puzzle has many solutions because it doesn't have to store completed puzzles and manage the storage growing depending on the number of solutions found.

Maintainability:

Both programs properly use classes and objects to organize the program, but the responsibilities are divided differently for each program. Claude uses Board to store the puzzle, read the file, check placement, and print the board. SudokuSolver handles the recursive search and stores completed solutions. PuzzleRunner manages the processing of a puzzle file and displays the results of the puzzle. This separation of responsibilities makes it easier to see which part of the program is doing what and what needs to be changed and updated for a specific responsibility.

Gemini uses its SudokuBoard for storing, loading, printing, and solving the puzzle. SudokuApp manages the five files and displays the results of the puzzles. This gives Gemini a shorter implementation with fewer classes. Printing the solutions immediately also reduces the clutter and avoids the additional vector management that Claude has. With this in mind, combining the board operations and recursive solving into one big class means that SudokuBoard has more responsibilities, which may make the class harder to maintain as more features are added. Its solveAll() method also prints to the terminal directly, so changing the way solutions are reported would mean having to make edits to the method for solving.

Both programs use useful naming conventions for methods and include comments that already explain main sections of the code. Both also keep board data private, which controls access to the board data from other parts of the program. For the board dimensions, Claude defines two constants called SIZE and BLOCK, while Gemini uses the numbers 9 and 3 repeatedly. Claude's naming convention for this makes it clearer without using repeated integers that may be confusing later on. Claude's implementation also allows for print() to accept an output stream, which makes it easier to direct the board output to the terminal or a file if desired.

Overall, both programs are readable and efficient, but the way Claude separates the responsibilities makes it easier to understand and easier to see where modifications can be made for future changes. Gemini's implementation is slightly simpler since it uses two classes, but its solving and output operations are more closely connected, which can make changes less straightforward. Both programs also need more detailed comments as well as a complete prologue at the beginning.

f) Program selection and justification: Claude

I chose Claude's program as the basis for the assignment. Both programs successfully produced results for the provided puzzles, but Claude handles conflicting starting clues more reliably. Its givensAreConsistent() method checks to ensure the original digits are valid before starting its search; Gemini does not do this. In an additional test, Gemini accepted a completely filled board with conflicting digits as a solution, which shouldn't happen.

The way Claude separated responsibilities across Board, SudokuSolver, and PuzzleRunner was also preferable. Each class's purpose is clear, and it makes it easier to make edits and modifications with more knowledge of the scope of what is being changed. Claude also uses named constants for the dimensions of the board, which read more clearly to me instead of using repeated integers every time; its printing method also accepts an output stream. Gemini uses less memory because it prints each solution immediately, but Claude's solution storage can be improved while keeping its clearer class organization. Overall, Claude has a stronger starting point because of its clearly separated responsibilities and starting clue validation.

g) The following is a summary of the improvements made to the code

Correctness:

There were many changes made to the correctness of Claude's generated code. The load() method was updated to accept only digits 1 through 9 or underscores as cells while still allowing whitespace. Previously, the program treated any other non-whitespace character as an empty cell; this could cause any potential invalid input to be accepted. The revised method also makes sure to only accept exactly 81 cells and checks for any potential content after the board is filled. It rejects any puzzle that may be incomplete and checks why the reading was stopped, whether that is due to the file ending normally or a reading error occurring.

Loading-error messages were also added to give more detail as to why a file couldn't be loaded should that occur. These messages identify opening failures, invalid characters, extra content, incomplete puzzles, and file reading failures. The load() method gives an explanation through an errorMessage reference parameter, and PuzzleRunner shows it alongside the filename. If a failure occurs, the program continues processing any remaining files.

After the revisions were made, testing confirmed that the revised program still functions as expected for all the puzzles. Additional tests confirmed that any issues like invalid characters, extra cells, incomplete files, empty files, and missing files are denied. Another test that triggered the unexpected reading failure branch confirmed that remaining puzzle files continue to be processed even if a failure occurs.

Execution Time:

The findEmpty() and search() methods were updated to reduce repeatedly scanning the board. In the original program, every recursive call would start searching for an empty cell from the top left corner even though earlier cells were already filled. The findEmpty() revisions allow it to accept a startIndex parameter. After a candidate is placed, search() passes the position directly after that cell to the next recursive call. This helps with avoiding repeatedly checking earlier positions that no longer need to be checked.

The worst-case time complexity of the program remains at O(9^m), where m is the number of initially empty cells. The program continues to explore any possible digit combinations and continues to search even after a solution is found. However, the revised program reduces some unnecessary work that was happening within the search. In some additional testing, the number of cell inspections in findEmpty() for puzzle five decreased by a very large amount while producing the same output. This shows how much the reduced searching actually benefits the program.

The program was also changed to print solutions immediately, rather than storing them in a vector, which removes some work from the program. Every solution still needs to be printed but doesn't need to be stored in a vector first to be accessed later. This avoids copying each completed board into a vector and potentially having to reallocate the vector as it grows. These changes overall reduce scanning, copying, and allocation work without changing the overall complexity of the program.

Space Complexity:

The main improvement that was made to space complexity was removing the vector that stored solutions that were already completed. Claude's original program stored completed puzzles into a vector of Board objects, giving it a space complexity of O(m+s), with m being the number of initially empty cells and s being the number of solutions that were found. The revised program prints each solution right away instead of storing it in a vector and uses solutionCount to track how many solutions were found. This removes the need of the vector that grows with the number of solutions.

After revisions, the program has a space complexity of O(m) because of the removed solutions vector and there still being a recursion stack. Each recursive call fills one cell before continuing, which makes the maximum recursion depth dependent on how many empty cells need to be filled. The working board and solution counter use a fixed amount of space. The startIndex parameter and nextIndex variable also use up a fixed amount of space for each recursive call, so they don't change the overall O(m) space complexity.

The errorMessage string stores a loading-error description, and the number of solutions doesn't cause this to grow. The unused vector include was also removed, but this doesn't change the space complexity of the program. Overall, the program still accurately finds and prints every solution without having to grow a collection of completed boards as the original did.

Maintainability:

Several changes were made to make the program more maintainable and easier to digest. The unused get() method and vector include were removed to reduce unnecessary clutter. A separate printSolution() method was added to keep the formatting of the solution outside the recursive search steps. The solveAll() method now accepts an output stream and returns a solution count, which PuzzleRunner uses to report the final results.

The revised load() method provides an error description through errorMessage, while PuzzleRunner displays it. This allows for separation between identifying loading problems and reporting them, and allows failures to be more easily diagnosed.

Comments were also added to every line and major block of code to make the program clearer in its methodology. A complete prologue section was also added, as well as source attribution for the code. The revision list in the prologue lists the revisions made, and the source notes help distinguish original Claude code from modifications made to it. The program overall keeps its class structure while making the revised behavior easier to follow.

Other GenAI Assistance: ChatGPT was used for assistance with prompting, testing, analyzing the generated programs, and making improvements.